

Advanced Features Guide

Four New Features Added!

1. **Authentication** - Secure admin panel
 2. **Search** - Full-text search with instant results
 3. **Image Upload** - Easy image management
 4. **Comments** - Reader engagement system
-

1 Authentication System

What It Does

- JWT-based authentication
- Secure admin panel access
- Password hashing with bcrypt
- Token-based sessions

Setup

1. Create Admin User:

```
cd backend
cargo run --bin setup
```

This creates a default admin user: - Email: `admin@example.com` - Password: `password123`

2. **Login:** Visit `http://localhost:3001/login` and use the credentials above.

Features

Login Page: - Email/password authentication - Error handling - Automatic redirect to admin panel

Protected Routes: - `/admin/*` routes require authentication - Automatic token verification - Session persistence (`localStorage`)

API Endpoints:

POST <code>/api/auth/login</code>	- Login with email/password
GET <code>/api/auth/verify</code>	- Verify JWT token

Usage

Frontend (React):

```
import { useAuth } from '@contexts/AuthContext'

function MyComponent() {
```

```

const { user, isAuthenticated, login, logout } = useAuth()

// Login
await login('admin@example.com', 'password123')

// Logout
logout()

// Check auth
if (isAuthenticated) {
  // User is logged in
}
}

```

API Requests with Auth:

```

const token = localStorage.getItem('auth_token')
fetch('/api/posts', {
  headers: {
    'Authorization': `Bearer ${token}`
  }
})

```

Security Notes

Current Implementation: - JWT tokens expire after 7 days - Passwords hashed with bcrypt (cost 12) - Tokens stored in localStorage

For Production: - [] Add refresh tokens - [] Implement rate limiting - [] Add 2FA support - [] Use HTTP-only cookies instead of localStorage - [] Add CSRF protection

2 Search Functionality

What It Does

- Full-text search using SQLite FTS5
- Instant search results
- Keyboard shortcuts (K / Ctrl+K)
- Search in titles, content, and excerpts

Features

Search Bar: - Click or press K to open - Type-ahead suggestions - Debounced queries (300ms) - Shows results while typing

Search Results: - Post title - Excerpt - Creation date - View count - Click to navigate

Keyboard Shortcuts: - K or Ctrl+K - Open search - ESC - Close search - Enter - Navigate to selected result

API Endpoint

GET /api/search?q=keyword

Response:

```
{
  "success": true,
  "data": [
    {
      "id": "...",
      "title": "Matching Post",
      "slug": "matching-post",
      "excerpt": "...",
      ...
    }
  ]
}
```

How It Works

Backend: 1. Creates FTS5 virtual table 2. Triggers keep FTS in sync with posts 3. Ranks results by relevance 4. Only searches published posts

Frontend: 1. Debounces input (prevents spam) 2. Queries API when 2+ characters 3. Displays results instantly 4. Highlights in modal

Customization

Change minimum search length:

```
// In SearchBar.tsx
if (debouncedQuery.length < 2) // Change 2 to your preference
```

Change debounce delay:

```
const debouncedQuery = useDebounce(query, 300) // Change 300ms
```

Limit results:

```
// In repository.rs search method
.search(&params.q, 20) // Change 20 to your limit
```

3 Image Upload

What It Does

- Upload images for blog posts
- Base64 encoding
- File validation (type, size)
- Automatic file naming
- Stores in /uploads/images

API Endpoint

POST /api/upload/image

Request:

```
{
  "filename": "image.jpg",
  "content_type": "image/jpeg",
  "data": "base64_encoded_data_here"
}
```

Response:

```
{
  "success": true,
  "data": {
    "url": "http://localhost:3000/uploads/images/1234567890-uuid.jpg",
    "filename": "1234567890-uuid.jpg"
  }
}
```

Validation

File Types Allowed: - image/jpeg - image/png - image/gif - image/webp

Size Limit: - Maximum: 5MB per image

Naming: - Format: {timestamp}-{uuid}.{ext} - Example: 1706745600-a1b2c3d4.jpg

Usage Example

Frontend (React):

```
const handleImageUpload = async (file: File) => {
  const reader = new FileReader()
  reader.onload = async (e) => {
    const base64 = e.target?.result?.toString().split(',')[1]

    const response = await fetch('/api/upload/image', {
      method: 'POST',
```

```

    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      filename: file.name,
      content_type: file.type,
      data: base64
    })
  })

  const result = await response.json()
  console.log('Image URL:', result.data.url)
}
reader.readAsDataURL(file)
}

```

In Post Form:

```

<input
  type="file"
  accept="image/*"
  onChange={(e) => {
    const file = e.target.files?.[0]
    if (file) handleImageUpload(file)
  }}
/>

```

Storage

Directory Structure:

```

backend/
  uploads/
    images/
      1706745600-uuid1.jpg
      1706745601-uuid2.png
      ...

```

Static File Serving: Images are served at: `http://localhost:3000/uploads/images/{filename}`

For Production

Recommended Changes: 1. Use cloud storage (S3, Cloudinary, Vercel Blob)
2. Add image optimization 3. Generate thumbnails 4. CDN for delivery

Vercel Blob Example:

```

import { put } from '@vercel/blob'

const blob = await put('images/photo.jpg', file, {
  access: 'public',

```

```
}  
// blob.url = 'https://...'
```

4 Comment System

What It Does

- Reader comments on posts
- Moderation workflow
- Spam prevention
- Email collection for notifications

Features

For Readers: - Leave comments on posts - Name and email required - Comments pending approval - See approved comments

For Admins: - Approve/reject comments - Delete spam - View pending count - Moderate all comments

Comment Statuses

1. **Pending** - Newly submitted, awaiting approval
2. **Approved** - Visible to everyone
3. **Spam** - Rejected as spam

API Endpoints

POST /api/posts/{post_id}/comments	- Submit comment
GET /api/posts/{post_id}/comments	- Get approved comments
PUT /api/comments/{id}/approve	- Approve comment (admin)
DELETE /api/comments/{id}	- Delete comment (admin)
GET /api/admin/comments/pending	- Get pending count (admin)

Comment Form

Required Fields: - Name - Email - Comment text

Validation: - All fields required - Email format validation - Content not empty

Moderation Workflow

1. Reader submits comment → Status: Pending
2. Admin reviews in dashboard
3. Admin approves → Status: Approved (visible)
4. OR Admin rejects → Status: Spam (hidden)

Frontend Component

Usage:

```
import Comments from '@components/Comments'
```

```
<Comments postId={post.id} />
```

Features: - Comment form - Comment list - Loading states - Error handling - Success messages

Customization

Auto-approve comments:

```
// In comment_repository.rs  
CommentStatus::Approved // Instead of Pending
```

Change form fields: Edit frontend/components/Comments.tsx

Add spam detection: Integrate Akismet or similar service

Email Notifications (Future)

Setup with Resend:

```
import { Resend } from 'resend'  
  
const resend = new Resend(process.env.RESEND_API_KEY)  
  
await resend.emails.send({  
  from: 'blog@yourdomain.com',  
  to: 'admin@example.com',  
  subject: 'New Comment',  
  html: `

New comment on "${post.title}"

`  
})
```

Complete Setup Guide

1. Backend Setup

```
cd backend  
  
# Install dependencies  
cargo build  
  
# Create admin user  
cargo run --bin setup
```

```
# Start server
```

```
cargo run
```

2. Frontend Setup

```
cd frontend
```

```
# Install dependencies
```

```
npm install
```

```
# Start dev server
```

```
npm run dev
```

3. Test All Features

Authentication: 1. Visit <http://localhost:3001/login> 2. Login with `admin@example.com / password123` 3. Access admin panel

Search: 1. Click search bar or press K 2. Type search query 3. See instant results

Image Upload: 1. Go to admin → Create Post 2. Upload featured image 3. See preview

Comments: 1. Visit any blog post 2. Leave a comment 3. See it pending approval 4. Approve in admin panel (coming soon)

New Database Schema

Users Table:

```
CREATE TABLE users (  
  id TEXT PRIMARY KEY,  
  email TEXT UNIQUE NOT NULL,  
  username TEXT UNIQUE NOT NULL,  
  password_hash TEXT NOT NULL,  
  full_name TEXT,  
  bio TEXT,  
  avatar_url TEXT,  
  role TEXT NOT NULL DEFAULT 'author',  
  created_at INTEGER NOT NULL,  
  updated_at INTEGER NOT NULL  
);
```

Comments Table:

```
CREATE TABLE comments (  
  id TEXT PRIMARY KEY,
```

```

    post_id TEXT NOT NULL,
    author_name TEXT NOT NULL,
    author_email TEXT NOT NULL,
    content TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'pending',
    created_at INTEGER NOT NULL,
    FOREIGN KEY (post_id) REFERENCES posts(id)
);

```

FTS Table:

```

CREATE VIRTUAL TABLE posts_fts USING fts5(
    title,
    content,
    excerpt
);

```

Security Checklist

Before Production:

- ☐ Change default admin password
 - ☐ Set strong JWT_SECRET in .env
 - ☐ Enable HTTPS only
 - ☐ Add rate limiting
 - ☐ Implement CSRF protection
 - ☐ Use HTTP-only cookies
 - ☐ Add refresh tokens
 - ☐ Enable 2FA
 - ☐ Sanitize user inputs
 - ☐ Add API request logging
-

UI Customization

Search Modal Colors: Edit frontend/components/SearchBar.tsx

Comment Form Style: Edit frontend/components/Comments.tsx

Login Page: Edit frontend/app/login/page.tsx

Tips & Best Practices

Authentication

- Change default password immediately

- Use environment variables for secrets
- Implement password reset flow
- Add email verification

Search

- Keep search queries short
- Use meaningful keywords
- Index important content only
- Monitor search analytics

Image Upload

- Compress images before upload
- Use descriptive filenames
- Add alt text for accessibility
- Consider lazy loading

Comments

- Moderate regularly
- Respond to comments
- Set clear guidelines
- Block spam domains

Troubleshooting

“Admin user already exists”

```
# Reset database
turso db destroy blog-db
turso db create blog-db
cargo run --bin setup
```

Search not working

```
# Rebuild FTS index
# In Turso CLI:
DELETE FROM posts_fts;
INSERT INTO posts_fts SELECT rowid, title, content, excerpt FROM posts;
```

Images not showing

```
# Check uploads directory exists
mkdir -p backend/uploads/images
```

```
# Check file permissions
chmod 755 backend/uploads/images
```

Comments not appearing

- Check if approved (status = 'approved')
 - Verify post_id is correct
 - Check API response in browser console
-

API Documentation

Authentication

POST /api/auth/login

```
Request: { "email": "...", "password": "..." }
Response: {
  "success": true,
  "data": {
    "token": "jwt_token_here",
    "user": { "id": "...", "email": "...", ... }
  }
}
```

Search

GET /api/search?q=keyword

```
Response: {
  "success": true,
  "data": [{ "id": "...", "title": "...", ... }]
}
```

Image Upload

POST /api/upload/image

```
Request: {
  "filename": "image.jpg",
  "content_type": "image/jpeg",
  "data": "base64..."
}
Response: {
  "success": true,
  "data": { "url": "...", "filename": "..." }
}
```

Comments

POST /api/posts/{post_id}/comments

```
Request: {  
  "author_name": "John",  
  "author_email": "john@example.com",  
  "content": "Great post!"  
}  
Response: {  
  "success": true,  
  "data": { "id": "...", "status": "pending", ... }  
}
```

What's Next?

Potential Enhancements:

1. **Rich Text Editor** - WYSIWYG instead of markdown
2. **Email Notifications** - Notify on new comments
3. **Social Login** - Google, GitHub OAuth
4. **Analytics Dashboard** - View stats and metrics
5. **Newsletter** - Email subscriptions
6. **Categories** - Organize posts by category
7. **Related Posts** - Show similar content
8. **Draft Scheduling** - Schedule post publishing

Let me know what you'd like to build next!

Enjoy your new features!

Need help? Check the main README or create an issue.